



Sistemi Distribuiti

Laurea in Informatica

Prof. Michele Ciavotta

michele.ciavotta@unimib.it

RIA - AJAX

INTERAZIONE CON I SERVER

L'oggetto XMLHttpRequest

Cos'è

- API JavaScript fornita da tutti i browser moderni per inviare richieste HTTP e ricevere risposte **senza ricaricare la pagina**.
- È il "motore" dietro AJAX: incapsula socket, formattazione del messaggio HTTP e parsing della risposta.
- Il nome è storico e fuorviante: nasce per scaricare XML, ma oggi si usa per qualsiasi formato (HTML, JSON, testo, binario).

Cosa offre

- **Modello asincrono**: la richiesta non blocca la UI; alla risposta scatta un evento.
- **Ciclo di vita esposto**: la proprietà *readyState* indica a che punto è la richiesta (0–4).
- **API a callback**: si registra un handler su *onreadystatechange* che reagisce ai cambiamenti di stato.

L'API essenziale di XMLHttpRequest

Metodi principali

Metodo	Cosa fa
<code>new XMLHttpRequest()</code>	Crea l'oggetto richiesta
<code>open(method, url, async)</code>	Configura la richiesta (GET/POST, URL, asincrona)
<code>setRequestHeader(k, v)</code>	Aggiunge un header HTTP (es. Content-Type)
<code>send(body)</code>	Invia la richiesta. <code>body = null</code> per GET
<code>abort()</code>	Interrompe una richiesta in corso
<code>getResponseHeader(name)</code>	Legge un header HTTP della risposta

Proprietà di interesse

Proprietà	Significato
<code>readyState</code>	Stato del ciclo di vita (0–4)
<code>status</code>	Codice HTTP della risposta (es. 200, 404)
<code>responseText</code>	Corpo della risposta come stringa
<code>responseXML</code>	Corpo della risposta parsato come DOM XML
<code>onreadystatechange</code>	Handler chiamato a ogni cambio di stato
<code>statusText</code>	Descrizione testuale dello status (es. "OK", "Not Found")

Schema d'uso ricorrente: 1) creare l'oggetto, 2) registrare `onreadystatechange`, 3) `open()` con i parametri, 4) eventuali `setRequestHeader`, 5) `send()`.

`setRequestHeader` serve quando si invia un POST con dati form: imposta header HTTP come Content-Type prima di `send()`.

Sincrono o asincrono?

Il terzo parametro di `open(method, url, async)` sceglie la modalità di esecuzione.

`async = true` (default, raccomandato)

- Il browser invia la richiesta e **continua a eseguire** il resto dello script: la UI resta reattiva.
- La risposta arriverà più tardi: bisogna registrare un handler con `onreadystatechange` prima di chiamare `send()`.

`async = false` (sconsigliato)

- `send()` blocca il thread JavaScript fino alla risposta: la pagina si congela.
- Deprecato sul main thread; tutti i browser mostrano un warning. Da evitare in produzione.

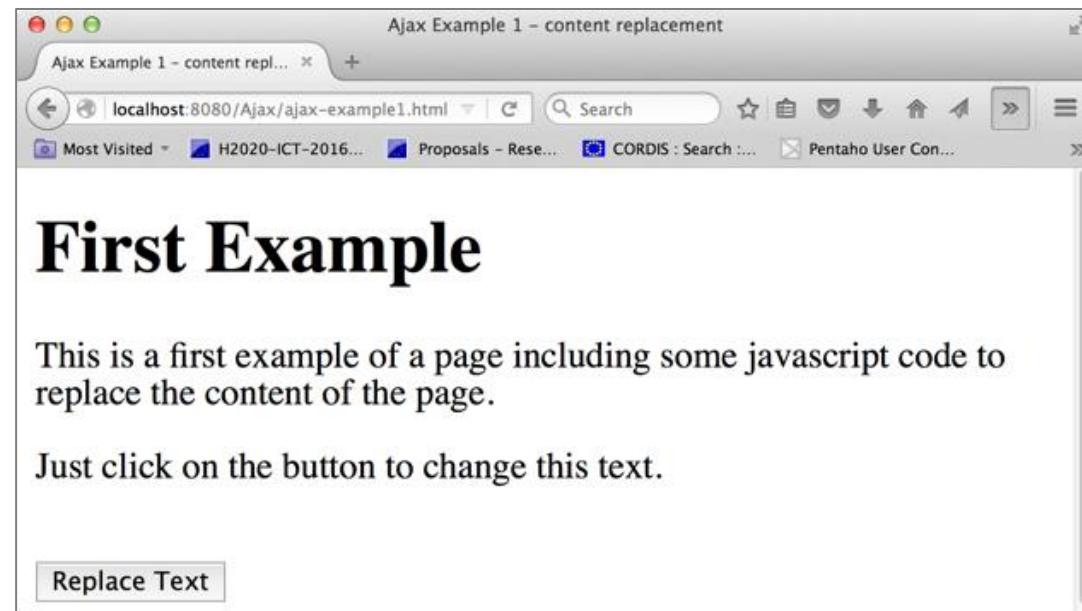
Conseguenza pratica: il modello di programmazione è event-driven. La risposta non si "ritorna", si "riceve".

Un primo esempio

```
1. <body>
2.   <div id="demo">
3.     <h1>First Example</h1>
4.     <p>This is a first example of a page including some javascript code to replace the content of the page.</p>
5.     <p>Just click on the button to change this text.</p>
6.   </div>
7.   <br>
8.   <button type="button" onclick="loadDoc()">Replace Text</button>
9.   <script>
10.    function loadDoc() ...
11.  </script>
12. </body>
```

Quando il pulsante viene cliccato, viene chiamata la funzione JavaScript `loadDoc()`

L'effetto è sostituire il testo nel div "demo"

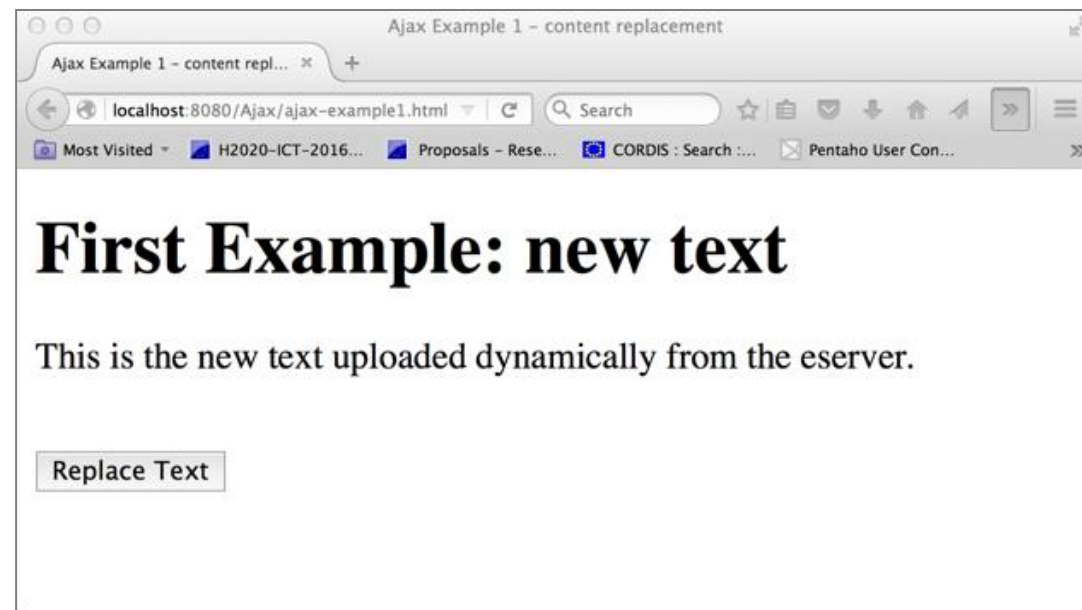


Un primo esempio... continua

```
1. <script>
2.   function loadDoc() {
3.     const xhttp = new XMLHttpRequest();
4.     xhttp.onreadystatechange = function() {
5.       if (xhttp.readyState == 4 && xhttp.status == 200) {
6.         document.getElementById("demo").innerHTML = xhttp.responseText;
7.       }
8.     };
9.     xhttp.open("GET", "ajax_text.html", true);
10.    xhttp.send();
11.  }
12. </script>
```

La funzione `loadDoc()`

1. carica il file `ajax_text.html`
2. se la nuova risorsa viene caricata correttamente (status code 200)
3. il testo ricevuto viene visualizzato come contenuto del div "demo"



Uso di una Callback Function

- Una funzione di callback è una funzione passata come parametro a un'altra funzione.
- La chiamata di funzione deve contenere l'URL e cosa fare in `onreadystatechange` (probabilmente diverso per ogni chiamata):

```
1.  function loadDoc(cFunc) {  
2.      const xhttp = new XMLHttpRequest();  
3.      xhttp.onreadystatechange = function() {  
4.          if (xhttp.readyState == 4 && xhttp.status == 200) {  
5.              cFunc(xhttp);  
6.          }  
7.      };  
8.      xhttp.open("GET", "ajax_text.html", true);  
9.      xhttp.send();  
10. }
```

Il parametro attuale sarà una
funzione per elaborare i dati
ricevuti dal server

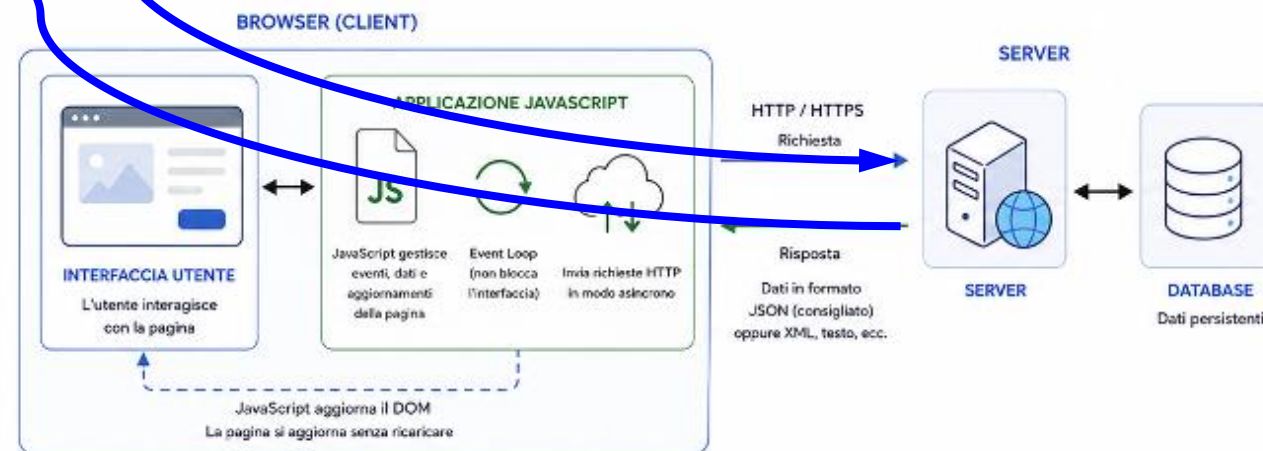
Chiama la funzione effettiva per
elaborare la risposta

Problema

- Domanda: Posso cambiare l'ordine delle istruzioni?

```
1. <script>
2.   function loadDoc() {
3.     var xhttp = new XMLHttpRequest();
4.     xhttp.open("GET", "ajax_text.html", true);
5.     xhttp.send();
6.     xhttp.onreadystatechange = function() {
7.       if (xhttp.readyState == 4 && xhttp.status == 200) {
8.         document.getElementById("demo").innerHTML = xhttp.responseText;
9.       }
10.    };
11.  }
12. </script>
```

- Risposta: No!
 - Perché il codice sopra è "sbagliato"?



Dietro le quinte di XMLHttpRequest

La classe XMLHttpRequest() definisce funzioni che nascondono le chiamate alla API di Sistema

È importante comprendere cosa accade (da un punto di vista logico) quando viene eseguita una di queste funzioni

```
1. <script>
2.   function loadDoc() {
3.       const xhttp = new XMLHttpRequest();
4.       xhttp.onreadystatechange = function() {
5.           if (xhttp.readyState == 4 && xhttp.status == 200) {
6.               document.getElementById("demo").innerHTML = xhttp.responseText;
7.           }
8.       };
9.       xhttp.open("GET", "ajax_text.html", true);
10.      xhttp.send();
11.  }
12. </script>
```

Crea l'oggetto di richiesta, mentre la gestione della connessione è delegata al browser.

Dietro le quinte di XMLHttpRequest

La classe XMLHttpRequest() definisce funzioni che nascondono le chiamate alla API di Sistema

È importante comprendere cosa accade (da un punto di vista logico) quando viene eseguita una di queste funzioni

```
1. <script>
2.   function loadDoc() {
3.     const xhttp = new XMLHttpRequest();
4.     xhttp.onreadystatechange = function() {
5.       if (xhttp.readyState == 4 && xhttp.status == 200) {
6.         document.getElementById("demo").innerHTML = xhttp.responseText;
7.       }
8.     };
9.     xhttp.open("GET", "ajax_text.html", true);
10.    xhttp.send();
11.  }
12. </script>
```

Aprire la connessione con la connect() e crea la prima riga di richiesta (obbligatoria) in formato HTTP

Può popolare l'header del messaggio con funzioni dedicate
NOTA: header di default vengono aggiunti automaticamente

Dietro le quinte di XMLHttpRequest

La classe XMLHttpRequest() definisce funzioni che nascondono le chiamate alla API di Sistema

È importante comprendere cosa accade (da un punto di vista logico) quando viene eseguita una di queste funzioni

```
1. <script>
2.   function loadDoc() {
3.     const xhttp = new XMLHttpRequest();
4.     xhttp.onreadystatechange = function() {
5.       if (xhttp.readyState == 4 && xhttp.status == 200) {
6.         document.getElementById("demo").innerHTML = xhttp.responseText;
7.       }
8.     };
9.     xhttp.open("GET", "ajax_text.html", true);
10.    xhttp.send();
11.  }
12. </script>
```

Utilizza onreadystatechange per segnalare un nuovo stato del ciclo di scrittura/lettura che il programma può conoscere attraverso la variabile readyState

Dietro le quinte di XMLHttpRequest

La classe XMLHttpRequest() definisce funzioni che nascondono le chiamate alla API di Sistema

È importante comprendere cosa accade (da un punto di vista logico) quando viene eseguita una di queste funzioni

```
1. <script>
2.   function loadDoc() {
3.     const xhttp = new XMLHttpRequest();
4.     xhttp.onreadystatechange = function() {
5.       if (xhttp.readyState == 4 && xhttp.status == 200) {
6.         document.getElementById("demo").innerHTML = xhttp.responseText;
7.       }
8.     };
9.     xhttp.open("GET", "ajax_text.html", true);
10.    xhttp.send();
11.  }
12. </script>
```

Chiude il messaggio, lo invia utilizzando `write()`, e avvia la lettura della risposta con `read()`
NOTA: invia senza body perché è una GET

Il ciclo di vita di una richiesta AJAX: la proprietà readyState

Il browser, tramite l'oggetto **XMLHttpRequest**, attraversa diverse fasi durante la comunicazione con il server. La proprietà **readyState** indica in quale fase ci troviamo.

Il flag che ci interessa è **readyState === 4** (richiesta completata).

Accoppiato a **status === 200** indica una risposta utilizzabile.

L'evento **onreadystatechange** viene scatenato a ogni transizione: il nostro handler tipicamente filtra solo lo stato 4.



Cos'è readyState?

readyState è una proprietà dell'oggetto XMLHttpRequest che rappresenta lo stato interno della richiesta HTTP.

Viene aggiornata automaticamente dal browser e il suo valore cambia ogni volta che la richiesta avanza in una nuova fase.



L'evento readystatechange

Ogni volta che readyState cambia, il browser scatena l'evento **readystatechange**. Possiamo intercettarlo per eseguire del codice al momento giusto.

I 5 STATI DELLA PROPRIETÀ readyState



Sarà vero?

```
<script>
function loadDoc() {
  const xhttp = new XMLHttpRequest();
  let s = "<h2>Status</h2>";
  xhttp.onreadystatechange = function() {
    s = s + "readyState = " + xhttp.readyState;
    s = s + "<br> ----- header = " + xhttp.getAllResponseHeaders();
    s = s + "<br> ----- status = " + xhttp.status;
    s = s + "<br> ----- response = " + xhttp.responseText;
    s = s + "<br>"

    if (xhttp.readyState == 4 && xhttp.status == 200) {
      document.getElementById("demo").innerHTML = xhttp.responseText;
    }
    document.getElementById("status").innerHTML = s;
  };
  xhttp.open("GET", "ajax_text.html", true);
  xhttp.send();
}
</script>
```

Cosa fa lo script

La funzione `loadDoc()` crea un oggetto `XMLHttpRequest` e invia una richiesta asincrona al server.

Gestione degli stati

L'handler `onreadystatechange` viene invocato a ogni cambio di stato e accumula nella stringa `s` i valori di `readyState`, `header`, `status` e `responseText`.

Risposta ricevuta

Quando `readyState == 4` e `status == 200`, la risposta viene inserita nell'elemento `"demo"`; il log degli stati viene mostrato in `"status"`.

Invio della richiesta

`open()` configura una GET asincrona verso `ajax_text.html`; `send()` la inoltra al server.

Un esempio di onreadystatechange

The onreadystatechange event & readyState

This is an example for onreadystatechange event & readyState.
Just click on the button to change this text, and monitor the script behaviour.

Replace Text

The onreadystatechange event & readyState

This is the new text uploaded dynamically from the eserver.

Replace Text

Status

```
readyState = 1
----- header =
----- status = 0
----- response =
readyState = 2
----- header = Server: Apache-Coyote/1.1 Accept-Ranges: bytes Etag: W/"62-1460556806000" Last-Modified:
Wed, 13 Apr 2016 14:13:26 GMT Content-Type: text/plain Content-Length: 62 Date: Wed, 13 Apr 2016 14:45:19 GMT
----- status = 200
----- response =
readyState = 3
----- header = Server: Apache-Coyote/1.1 Accept-Ranges: bytes Etag: W/"62-1460556806000" Last-Modified:
Wed, 13 Apr 2016 14:13:26 GMT Content-Type: text/plain Content-Length: 62 Date: Wed, 13 Apr 2016 14:45:19 GMT
----- status = 200
----- response = This is the new text uploaded dynamically from the eserver.
readyState = 4
----- header = Server: Apache-Coyote/1.1 Accept-Ranges: bytes Etag: W/"62-1460556806000" Last-Modified:
Wed, 13 Apr 2016 14:13:26 GMT Content-Type: text/plain Content-Length: 62 Date: Wed, 13 Apr 2016 14:45:19 GMT
----- status = 200
----- response = This is the new text uploaded dynamically from the eserver.
```


Risposta del server

Property	Description
responseText	get the response data as a string
responseXML	get the response data as XML data

Per ottenere la risposta da un server, usare la proprietà `responseText` o `responseXML` dell'oggetto `XMLHttpRequest`.

La proprietà `responseText`

- Se la risposta dal server non è XML, usare la proprietà `responseText`.
- La proprietà `responseText` restituisce la risposta come stringa.

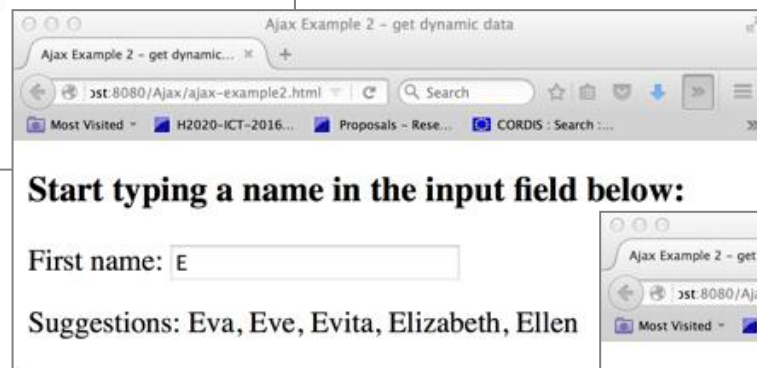
```
document.getElementById("demo").innerHTML = xhttp.responseText;
```

La proprietà `responseXML`

- Se la risposta dal server è XML e si vuole analizzarla come oggetto XML, usare la proprietà `responseXML`

L'esempio hint rivisto

- L'esempio seguente mostra come una pagina web può comunicare con un server web mentre l'utente digita caratteri in un campo di input
- L'interfaccia e l'interazione utente non cambiano



Il frontend HTML

- The HTML page is structured as in the client-only version of the example.
- Only the implementation of the `showHint(str)` function is different.

```
1. <html>
2. <head>
3. </head>
4. <body>

5. <h3>Start typing a name in the input field below:</h3>
6. <form action="">
7.   First name: <input type="text" id="txt1" onkeyup="showHint(this.value)">
8. </form>
9. <p>Suggestions: <span id="txtHint"></span></p>

10. <script>
11.   function showHint(str) ...
12. </script>

13. </body>
14. </html>
```

Il frontend HTML

```
1.  <script>
2.  function showHint(str) {
3.      // se il campo è vuoto, pulisco i suggerimenti
4.      if (str.length == 0) {
5.          document.getElementById("txtHint").innerHTML = "";
6.          return;
7.      }
8.      // creo la richiesta AJAX
9.      const xhttp = new XMLHttpRequest();
10.     // callback eseguita quando cambia lo stato della richiesta
11.     xhttp.onreadystatechange = function() {
12.         // richiesta completata con successo
13.         if (this.readyState == 4 && this.status == 200) {
14.             document.getElementById("txtHint").innerHTML = this.responseText;
15.         }
16.     };
17.     // invio richiesta GET alla servlet
18.     xhttp.open("GET", "GetHint?entry=" + str, true);
19.     xhttp.send();
20. }
21. </script>
```

La servlet backend

```
1. public class GetHint extends HttpServlet {
2.     private String[] names = {"Anna", "Brittany", "Cinderella", "Diana", "Eva", "Fiona", "Gunda", "Hege",
    "Inga", "Johanna", "Kitty", "Linda", "Nina", "Ophelia", "Petunia", "Amanda", "Eve", "Evita", "Sunniva",
    "Tove", "Unni", "Violet", "Liza", "Elizabeth", "Ellen", "Wenche", "Vicky"};

3.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
4.         throws ServletException, IOException {
5.         String entry = request.getParameter("entry");
6.         String hint = ""
7.         if (entry.length() > 0) { // lookup all hints from array if length of entry>0
8.             for (int i = 0; i < names.length; ++i) {
9.                 if (names[i].startsWith(entry)) {
10.                    if (hint.contentEquals("")) { hint = names[i]; }
11.                    else { hint = hint + ", " + names[i]; }
12.                } } }
13.         response.setContentType("text/plain");
14.         response.getWriter().append(hint);
15.     }
16. }
```

Conversione tra oggetti e JSON

- L'oggetto namespace JSON
 - Contiene metodi statici per analizzare valori da JSON e convertire valori in JSON
- `JSON.parse()`
 - Analizza una stringa di testo come JSON, opzionalmente trasformando il valore prodotto e le sue proprietà, e restituisce il valore

```
const jsonObj = '{ "name": "Jack (\\\"Bee\\\") Nimble", ' +  
  '"format": { "type": "rect", "width": 120, "interlace": false}}';  
const obj = JSON.parse(jsonObj);
```

- `JSON.stringify()`
 - Restituisce una stringa JSON corrispondente al valore specificato, opzionalmente includendo solo alcune proprietà o sostituendo i valori in modo personalizzato

```
const jsObj = { ... }  
const json = JSON.stringify(jsObj);
```

Un semplice esempio

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>JSON example</title>
</head>
<body>
<h2>JSON examples</h2>
<p id="output1">obj: </p>
<p id="output2">JSON: </p>

<script>
  const jsonObj = '{ "name": "Jack (\\"Bee\\") Nimble", ' +
    '"format": { "type": "rect", "width": 120, "interlace":
false}}';
  const obj = JSON.parse(jsonObj);
  const output1 = ' Il nome è ' + obj.name + ' di tipo ' +
    obj.format.type + ' e dimensione ' + obj.format.width;
  document.getElementById('output1').innerText += output1;

  const jsObj = {
    name: 'Jack ("Bee") Nimble',
    format: {
      type: "rect", width: 120, interlace: false
    }
  }
  const json = JSON.stringify(jsObj);
  document.getElementById('output2').innerText += ' ' + json;
</script>
</body>
</html>
```

Cosa fa lo script

La pagina mostra come passare dal formato testuale JSON al corrispondente oggetto JavaScript e viceversa, scrivendo i risultati nei due paragrafi `output1` e `output2`.

Da JSON a oggetto

La stringa `jsonObj` viene convertita in oggetto con `JSON.parse(...)`; le proprietà `obj.name`, `obj.format.type` e `obj.format.width` sono accessibili come campi nidificati e vengono concatenate in una frase descrittiva.

Da oggetto a JSON

L'oggetto JavaScript `jsObj`, definito direttamente nel codice, viene serializzato con `JSON.stringify(...)` nella stringa `json`, pronta per essere trasmessa o salvata.

Risultato in pagina

I valori prodotti vengono iniettati in pagina assegnandoli a `document.getElementById(...).innerText`.

Un semplice esempio

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>JSON example</title>
</head>
<body>
<h2>JSON examples</h2>
<p id="output1">obj: </p>
<p id="output2">JSON: </p>

<script>
  const jsonObj = '{ "name": "Jack (\\"Bee\\") Nimble", ' +
    '"format": { "type": "rect", "width": 120, "interlace":' +
false}}';
  const obj = JSON.parse(jsonObj);
  const output1 = ' Il nome è ' + obj.name + ' di tipo ' +
    obj.format.type + ' e dimensione ' + obj.format.width;
  document.getElementById('output1').innerText += output1;

  const jsonObj = {
    name: 'Jack ("Bee") Nimble',
    format: {
      type: "rect", width: 120, interlace: false
    }
  }
  const json = JSON.stringify(jsonObj);
  document.getElementById('output2').innerText += ' ' + json;
</script>
</body>
</html>
```



Oggetti JS da JSON

```
1. <!DOCTYPE html>
2. <html>
3. <head>
4. <meta charset="UTF-8">
5. <title>Object from String in JavaScript</title>
6. </head>
7. <body>
8. <h2>Create Object from JSON String</h2>
9. <p id="demo"></p>
10. <script>
11.   const text = '{ "employees": [' +
12.     '{ "firstName":"John", "lastName":"Doe" },' +
13.     '{ "firstName":"Anna", "lastName":"Smith" },' +
14.     '{ "firstName":"Peter", "lastName":"Jones" } ] }';
15.   const obj = JSON.parse(text);
16.   document.getElementById("demo").innerHTML =
17.     obj.employees[1].firstName + " " + obj.employees[1].lastName;
18. </script>
19. </body>
20. </html>
```

Cosa fa lo script

La pagina costruisce un oggetto JavaScript a partire da una stringa in formato JSON e ne mostra in pagina uno dei valori, scrivendolo dentro `<p id="demo">`.

La stringa JSON

La costante `text` contiene un JSON con una proprietà `employees`: un array di tre oggetti, ciascuno con i campi `firstName` e `lastName`.

Parsing in oggetto

La chiamata `JSON.parse(text)` trasforma la stringa in un oggetto JavaScript `obj` i cui campi sono accessibili con la notazione punto e l'indice di array.

Risultato in pagina

Lo script legge il secondo elemento dell'array (`obj.employees[1]`) e ne assegna nome e cognome a `document.getElementById("demo").innerHTML`, ottenendo "Anna Smith".

Oggetti JS da JSON

```
1. <!DOCTYPE html>
2. <html>
3. <head>
4. <meta charset="UTF-8">
5. <title>Object from String in JavaScript</title>
6. </head>
7. <body>
8. <h2>Create Object from JSON String</h2>
9. <p id="demo"></p>
10. <script>
11.   const text = '{ "employees": [' +
12.     '{ "firstName":"John", "lastName":"Doe" },' +
13.     '{ "firstName":"Anna", "lastName":"Smith" },' +
14.     '{ "firstName":"Peter", "lastName":"Jones" } ] }';
15.   const obj = JSON.parse(text);
16.   document.getElementById("demo").innerHTML =
17.     obj.employees[1].firstName + " " + obj.employees[1].lastName;
18. </script>
19. </body>
20. </html>
```

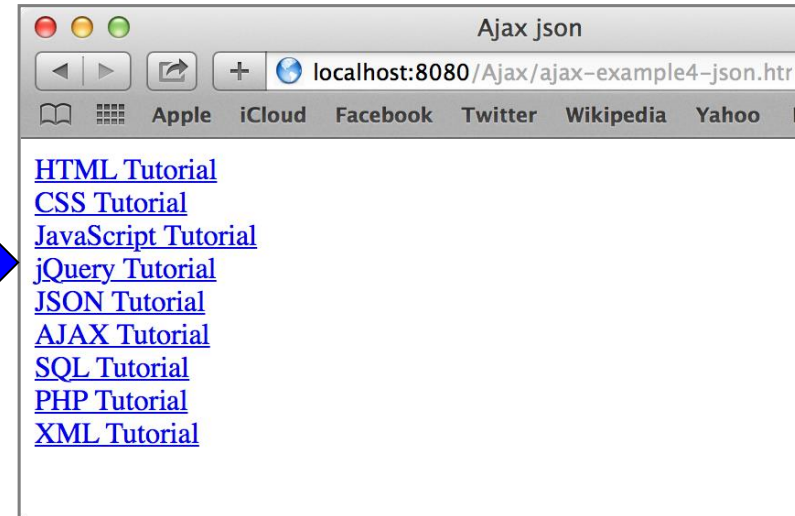
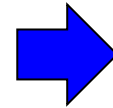


Un esempio: Oggetto da Stringa

- Recupera una lista di risorse Web (nome e URL) da un file JSON e la visualizza come lista HTML di riferimenti

myTutorials.json

```
[  
  { "display": "HTML Tutorial",  
    "url": "http://www.w3schools.com/html/default.asp" },  
  { "display": "CSS Tutorial",  
    "url": "http://www.w3schools.com/css/default.asp" },  
  { "display": "JavaScript Tutorial",  
    "url": "http://www.w3schools.com/js/default.asp" },  
  { "display": "jQuery Tutorial",  
    "url": "http://www.w3schools.com/jquery/default.asp" },  
  { "display": "JSON Tutorial",  
    "url": "http://www.w3schools.com/json/default.asp" },  
  { "display": "AJAX Tutorial",  
    "url": "http://www.w3schools.com/ajax/default.asp" },  
  { "display": "SQL Tutorial",  
    "url": "http://www.w3schools.com/sql/default.asp" },  
  { "display": "PHP Tutorial",  
    "url": "http://www.w3schools.com/php/default.asp" },  
  { "display": "XML Tutorial",  
    "url": "http://www.w3schools.com/xml/default.asp" }  
]
```



Ajax - Object From String

```
1. <body>
2. <div id="id01"></div>
3. <script>
4. const xmlhttp = new XMLHttpRequest();
5. const url = "myTutorials.json";

6. xmlhttp.onreadystatechange = function() {
7.     if (xmlhttp.readyState == 4 && xmlhttp.status == 200) {
8.         var myArr = JSON.parse(xmlhttp.responseText);
9.         myFunction(myArr);
10.    }
11. };
12. xmlhttp.open("GET", url, true);
13. xmlhttp.send();

14. function myFunction(arr) {
15.     let out = "";
16.     for(var i = 0; i < arr.length; i++) {
17.         out += '<a href="' + arr[i].url + '>' + arr[i].display + '</a><br>';
18.     }
19.     document.getElementById("id01").innerHTML = out;
20. }
21. </script>
22. </body>
```

Cosa fa lo script

Recupera `myTutorials.json` dal server, lo converte in oggetto JS e lo mostra come lista di link nel `<div id="id01">`.

Richiesta e parsing

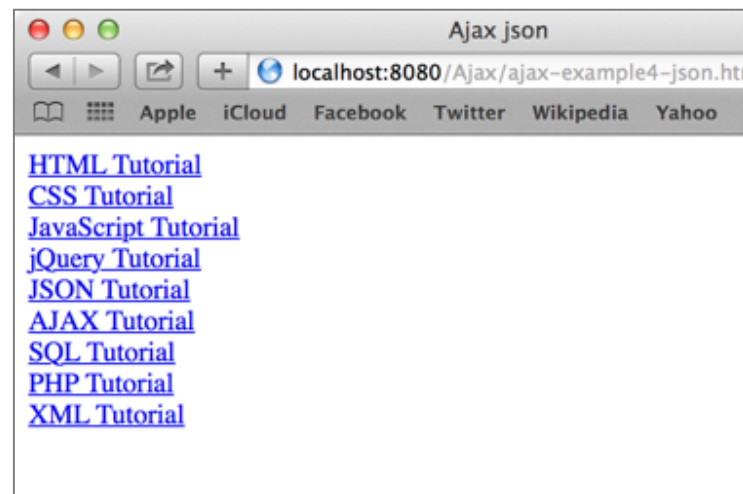
Una GET asincrona scarica il JSON; con `readyState == 4` e `status == 200`, `JSON.parse()` trasforma la stringa in array di oggetti.

Costruzione dell'output

`myFunction(arr)` scorre l'array e per ogni elemento costruisce un tag `<a>` con le proprietà `url` e `display`.

Risultato in pagina

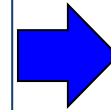
L'HTML viene assegnato a `innerHTML` del div, mostrando l'elenco dei link.



Un altro esempio: tabella formattata

customers.json

```
[ { "Name": "Alfreds Futterkiste", "City": "Berlin", "Country":  
"Germany" }, { "Name": "Berglunds snabbköp", "City":  
"Luleå", "Country": "Sweden" }, { "Name": "Centro comercial  
Moctezuma", "City": "México D.F.", "Country": "Mexico" }, {  
"Name": "Ernst Handel", "City": "Graz", "Country": "Austria"  
}, { "Name": "FISSA Fabrica Inter. Salchichas S.A.", "City":  
"Madrid", "Country": "Spain" }, { "Name": "Galería del  
gastrónomo", "City": "Barcelona", "Country": "Spain" }, {  
"Name": "Island Trading", "City": "Cowes", "Country": "UK"  
}, { "Name": "Königlich Essen", "City": "Brandenburg",  
"Country": "Germany" }, { "Name": "Laughing Bacchus Wine  
Cellars", "City": "Vancouver", "Country": "Canada" }, {  
"Name": "Magazzini Alimentari Riuniti", "City": "Bergamo",  
"Country": "Italy" }, { "Name": "North/South", "City":  
"London", "Country": "UK" }, { "Name": "Paris spécialités",  
"City": "Paris", "Country": "France" }, { "Name": "Rattlesnake  
Canyon Grocery", "City": "Albuquerque", "Country": "USA" },  
{ "Name": "Simons bistro", "City": "København", "Country":  
"Denmark" }, { "Name": "The Big Cheese", "City": "Portland",  
"Country": "USA" }, { "Name": "Vaffeljernet", "City": "Århus",  
"Country": "Denmark" }, { "Name": "Wolski Zajazd", "City":  
"Warszawa", "Country": "Poland" } ]
```



Ajax json CSS

/Ajax/ajax-example4-json-css.html

Reader

Apple iCloud Facebook Twitter Wikipedia

Customers

Alfreds Futterkiste	Berlin	Germany
Berglunds snabbköp	Luleå	Sweden
Centro comercial Moctezuma	México D.F.	Mexico
Ernst Handel	Graz	Austria
FISSA Fabrica Inter. Salchichas S.A.	Madrid	Spain
Galería del gastrónomo	Barcelona	Spain
Island Trading	Cowes	UK
Königlich Essen	Brandenburg	Germany
Laughing Bacchus Wine Cellars	Vancouver	Canada
Magazzini Alimentari Riuniti	Bergamo	Italy
North/South	London	UK
Paris spécialités	Paris	France
Rattlesnake Canyon Grocery	Albuquerque	USA
Simons bistro	København	Denmark
The Big Cheese	Portland	USA
Vaffeljernet	Århus	Denmark
Wolski Zajazd	Warszawa	Poland

... creare la tabella, e ...

```
1. <body>
2. <h1>Customers</h1>
3. <div id="id01"></div>
4. <script>
5.   const xmlhttp = new XMLHttpRequest();
6.   let url = "customers.json";
7.   xmlhttp.onreadystatechange=function() {
8.     if (xmlhttp.readyState == 4 && xmlhttp.status == 200) {
9.       myFunction(xmlhttp.responseText);
10.    }
11.  }
12.  xmlhttp.open("GET", url, true);
13.  xmlhttp.send();

14.  function myFunction(response) {
15.    const list = JSON.parse(response);
16.    let out = "<table>";
33.    for (let customer of list) {
34.      out += "<tr>";
35.      for (let field in customer) {
36.        out += "<td>" + customer[field] + "</td>";
37.      }
38.      out += "</tr>";
39.    }
40.    out += "</table>";
41.    document.getElementById("id01").innerHTML = out;
42.  }
43. </script>
44. </body>
```



Alfreds Futterkiste	Berlin	Germany
Berglunds snabbköp	Luleå	Sweden
Centro comercial Moctezuma	México D.F.	Mexico
Ernst Handel	Graz	Austria
FISSA Fabrica Inter. Salchichas S.A.	Madrid	Spain
Galería del gastrónomo	Barcelona	Spain
Island Trading	Cowes	UK
Königlich Essen	Brandenburg	Germany
Laughing Bacchus Wine Cellars	Vancouver	Canada
Magazzini Alimentari Riuniti	Bergamo	Italy
North/South	London	UK
Paris spécialités	Paris	France
Rattlesnake Canyon Grocery	Albuquerque	USA
Simons bistro	København	Denmark
The Big Cheese	Portland	USA
Vaffeljernet	Århus	Denmark
Wolski Zajazd	Warszawa	Poland

Suggerimento: inserire la riga di intestazione

L'esempio survey rivisto

Survey: favorite pet

What is your favorite pet?

☐ Dog
☐ Cat
☐ Bird
☐ Snake
☒ None

Survey: favorite pet

What is your favorite pet?

☐ Dog
☐ Cat
☒ Bird
☐ Snake
☐ None

Name	Percentage	Responses
dog	21.31	13
cat	9.84	6
bird	24.59	15
snake	14.75	9
none	29.51	18

Total responses: 61

surveyJson.html

```
1.  <!DOCTYPE html>
2.  <html>
3.  <body>
4.    <h1>Survey: favorite pet</h1>
5.    <form id="myform" action="">
6.      What is your favorite pet?<br> <br> <input type="radio"
7.      name="animal" value="dog">Dog<br> <input type="radio"
8.      name="animal" value="cat">Cat<br> <input type="radio"
9.      name="animal" value="bird">Bird<br> <input type="radio"
10.     name="animal" value="snake">Snake<br> <input
11.     type="radio" name="animal" value="none" checked>None <br>
12.     <br>
13.     <button type="button" onclick="loadDoc()">Send vote</button>
14.     <input type="reset">
15.   </form>
16.   <p id="demo">
17.   </p>
18. <script>
19.   function loadDoc() ...
20. </script>
21. </body>
22. </html>
```

Survey: favorite pet

What is your favorite pet?

- ☐ Dog
☐ Cat
☐ Bird
☐ Snake
☒ None

Send vote

Reset

Survey script (1)

```
1. <script>
2. function loadDoc() {
3.     let formElements = document.getElementById("myform").elements;
4.     let animal = "";
5.     for (i = 0; i < formElements.length ;i++) {
6.         if (formElements[i].checked) {
7.             animal = formElements[i].value;
8.             break;
9.         }
10.    }
11.    const xhttp = new XMLHttpRequest();
12.    xhttp.onreadystatechange = function() {
13.        if (xhttp.readyState == 4 && xhttp.status == 200) {
14.            createTable(xhttp.responseText);
15.        }
16.    };
17.    xhttp.open("POST", "SurveyJson", true);
18.    xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
19.    xhttp.send("animal=" + animal);
20.    return false;
21. }
```

Il content type per usare dati "form" HTML.
Il body è formattato come coppie
nome=valore

Survey script (2)

```
1. <script>
2. function loadDoc() ...

1. function createTable(response) {
2.     const arr = JSON.parse(response);
3.     let totalRes = 0;
4.     let out = "<table>";
5.     out += "<tr><td>Name</td><td>Percentage</td><td>Responses</td></tr>"
6.     for (let i = 0; i < arr.length; i++) {
7.         out += "<tr><td>" + arr[i].Name + "</td><td>"
8.             + arr[i].Percentage + "</td><td>" + arr[i].Responses
9.             + "</td></tr>";
10.        totalRes = totalRes + parseInt(arr[i].Responses);
11.    }
12.    out += "</table>";
13.    out += "<p> Total responses: " + totalRes + "</p>"
14.    document.getElementById("demo").innerHTML = out;
15. }

16. </script>
```

The JSON representation:

```
[ { "Name" : "dog",
  "Percentage" : "31.25" ,
  "Responses" : "5" }, {
  "Name" : "cat",
  "Percentage" : "6.25" ,
  "Responses" : "1" }, {
  "Name" : "bird",
  "Percentage" : "18.75" ,
  "Responses" : "3" }, {
  "Name" : "snake",
  "Percentage" : "31.25" ,
  "Responses" : "5" }, {
  "Name" : "none",
  "Percentage" : "12.50" ,
  "Responses" : "2" } ]
```

La servlet SurveyJson

```
1. @WebServlet("/SurveyJson")
2. public class SurveyJson extends HttpServlet {
3.     private static final long serialVersionUID = 1L;
4.     private String animalNames[] = { "dog", "cat", "bird", "snake", "none" };
5.
6.     /**
7.      * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response)
8.      */
9.     protected void doGet(HttpServletRequest request,
10.                          HttpServletResponse response)
11.         throws ServletException, IOException {
12.         ...
13.     }
14.
15.     /**
16.      * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse response)
17.      */
18.     protected void doPost(HttpServletRequest request,
19.                          HttpServletResponse response)
20.         throws ServletException, IOException {
21.         ...
22.     }
23. }
```

Il metodo doGet (1)

```
1. protected void doGet(HttpServletRequest request, HttpServletResponse response)
2.     throws ServletException, IOException {
3.     int animals[] = null, total = 0;
4.     File f = new File("survey.dat");
5.     if (f.exists()) { // Determine # of survey responses so far
6.         try {
7.             ObjectInputStream input = new ObjectInputStream(new FileInputStream(f));
8.
9.             animals = (int[]) input.readObject();
10.            input.close(); // close stream
11.
12.            for (int i = 0; i < animals.length; ++i)
13.                total += animals[i];
14.        } catch (ClassNotFoundException cnfe) {
15.            cnfe.printStackTrace();
16.        }
17.    } else
18.        animals = new int[5];
19.
20.    // Calculate percentages
21.    double percentages[] = new double[animals.length];
22.    for (int i = 0; i < percentages.length; ++i)
23.        percentages[i] = 100.0 * animals[i] / total;
```

Il metodo doGet (2)

```
21. // Send a JSON message
22. response.setContentType("application/json"); // content type
23. response.setCharacterEncoding("UTF-8");
24. PrintWriter responseOutput = response.getWriter();
25. StringBuffer buf = new StringBuffer();
26. buf.append("[ ");
27. DecimalFormat twoDigits = new DecimalFormat("#0.00");
28. for (int i = 0; i < percentages.length; ++i) {
29.     if (i != 0)
30.         buf.append(", ");
31.     buf.append("{ \"Name\" : \"");
32.     buf.append(animals[i]);
33.     buf.append("\", \"Percentage\" : \"");
34.     buf.append(twoDigits.format(percentages[i]));
35.     buf.append("\", \"Responses\" : \"");
36.     buf.append(animals[i]);
37.     buf.append("\", \"Responses\" : \"");
38.     buf.append(animals[i]);
39.     buf.append("\", \"Responses\" : \"");
40.     buf.append(animals[i]);
41.     buf.append("\", \"Responses\" : \"");
42.     buf.append(animals[i]);
```

The JSON representation:

```
[ { "Name" : "dog",
  "Percentage" : "31.25" ,
  "Responses" : "5" }, {
  "Name" : "cat",
  "Percentage" : "6.25" ,
  "Responses" : "1" }, {
  "Name" : "bird",
  "Percentage" : "18.75" ,
  "Responses" : "3" }, {
  "Name" : "snake",
  "Percentage" : "31.25" ,
  "Responses" : "5" }, {
  "Name" : "none",
  "Percentage" : "12.50" ,
  "Responses" : "2" } ]
```

Il metodo doPost

```
1. protected void doPost(HttpServletRequest request, HttpServletResponse response)
2.     throws ServletException, IOException {

3.     int animals[] = null;

4.     File f = new File("survey.dat");

5.     if (f.exists()) {
6.         // Determine # of survey responses so far
7.         try {
8.             ObjectInputStream input = new ObjectInputStream(new FileInputStream(f));

9.             animals = (int[]) input.readObject(); // load existing responses
10.            input.close(); // close stream

11.        } catch (ClassNotFoundException cnfe) {
12.            cnfe.printStackTrace();
13.        }
14.    } else
15.        animals = new int[5]; // first execution, no responses yet
```

Il metodo doPost

```
16. // read current survey response
17. String value = request.getParameter("animal");

18. // determine which was selected and update its total
19. for (int i = 0; i < animalNames.length; ++i)
20.     if (value.equals(animalNames[i]))
21.         ++animals[i];

22. // write updated totals out to disk
23. ObjectOutputStream output = new
    ObjectOutputStream(new FileOutputStream(f));

24. output.writeObject(animals);
25. output.flush();
26. output.close();

27.
28. // delegate doGet to provide the answer
29. doGet(request, response);
30. }
```

Survey: favorite pet

What is your favorite pet?

- ☐ Dog
☐ Cat
☒ Bird
☐ Snake
☐ None

Send vote

Reset

Name Percentage Responses

dog	21.31	13
cat	9.84	6
bird	24.59	15
snake	14.75	9
none	29.51	18

Total responses: 61

fetch: la stessa cosa, in meno righe

fetch() è l'API moderna per richieste HTTP. Restituisce una Promise: niente readyState, niente onreadystatechange. Disponibile in tutti i browser moderni e in Node.js dalla 18.

XMLHttpRequest

```
const xhr = new XMLHttpRequest();
xhr.onreadystatechange = function() {
  if (xhr.readyState === 4 &&
      xhr.status === 200) {
    const data = JSON.parse(xhr.responseText);
    use(data);
  }
};
xhr.open("GET", "/api/data", true);
xhr.send();
```

fetch + async/await

```
async function load() {
  try {
    const res = await fetch("/api/data");
    if (!res.ok) throw new Error(res.status);
    const data = await res.json();
    use(data);
  } catch (err) {
    console.error(err);
  }
}
```

Vantaggi: no callback hell, gestione errori con try/catch, parsing JSON nativo via res.json(), supporto a streaming, AbortController per cancellare richieste. È lo standard de-facto oggi: usate fetch nei vostri progetti.

AJAX: criticità note

Navigazione e bookmark

- Il tasto "back" e i preferiti registrano cambi di URL: con AJAX la pagina cambia contenuto senza cambiare URL, e l'utente perde il riferimento allo stato
- Mitigato dalla **History API** (pushState / popstate, dal 2010): permette di aggiornare URL e cronologia senza ricaricare

Indicizzazione e SEO

- I contenuti caricati via AJAX possono non essere indicizzati: storicamente i crawler non eseguivano JavaScript; oggi lo fanno ma con limiti
- Soluzioni moderne: **Server-Side Rendering (SSR)** e **idratazione**

Complessità del client

- La logica si sposta sul client: il bundle JavaScript cresce, allungando il tempo di *first load*
- Debug più difficile: la stessa logica è distribuita tra client e server

Pressione sul server

- AJAX rende facile fare molte richieste piccole e frequenti (polling, autocompletamento ad ogni tasto): l'effetto cumulativo può pesare più del singolo full reload

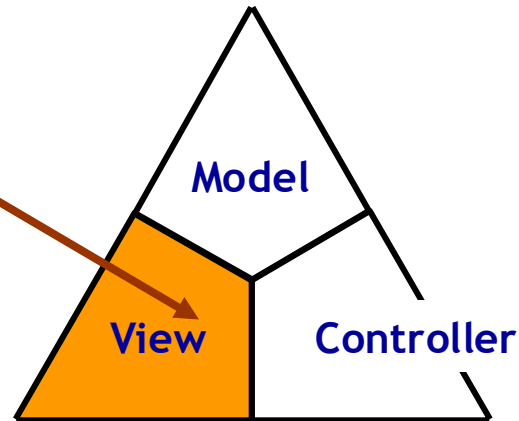
Rich Internet Apps: Il grande cambiamento

Web Browser

Server Centric

Static
HTMLPages

Web Server



Modello tradizionale

Tutta la logica applicativa — **Model, View e Controller** — vive sul **web server**.

Il browser è un terminale passivo: riceve **pagine HTML statiche** già pronte e si limita a visualizzarle.

Conseguenze

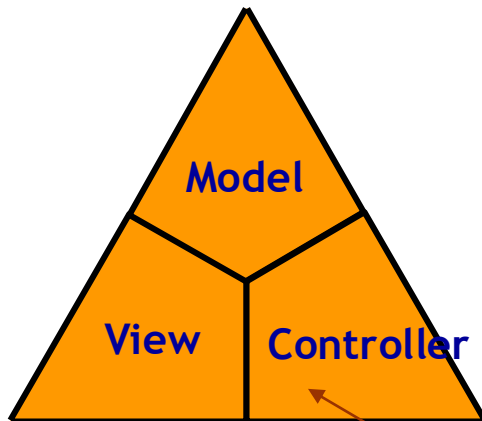
- Ogni interazione = nuova richiesta + ricaricamento dell'intera pagina.
- UX a scatti, traffico di rete elevato, server sotto carico.

Nella prossima slide vedremo come le RIA ribaltano questo schema.

RIA: Elaborazione lato client

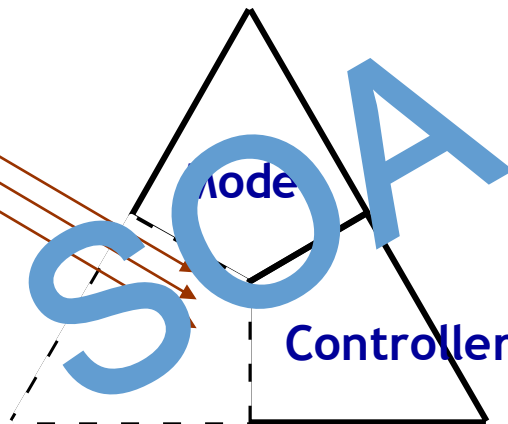
Web Browser

RIA



Client Centric

Web Server



Il ribaltamento

Una **porzione di Model e Controller** migra nel browser insieme alla View: è questo che rende l'applicazione **ricca**.

Il server resta autoritativo (dati, business logic), ma espone **servizi** (SOA): si scambiano **dati**, non pagine intere.

Vantaggi

- Aggiornamenti parziali, senza ricaricare la pagina.
- Logica nel client → UX fluida, simile a un'app desktop.
- Meno traffico, server alleggerito.

Serve un meccanismo per parlare col server in background: è AJAX.

Riferimenti

- Alcuni esempi sono derivati da:
<http://www.w3schools.com/Ajax/default.asp>
- Maggiori informazioni su JavaScript su:
<http://www.w3schools.com/js/default.asp>
- Dettagli sugli eventi su:
http://www.w3schools.com/js/js_events_examples.asp
- Risorse Mozilla
<https://developer.mozilla.org/en-US/docs/Web>
- Fonti in italiano
<http://www.html.it/guide/guida-ajax/>
<http://www.html.it/guide/guida-javascript-di-base/>
- CSS
https://developer.mozilla.org/en-US/docs/Web/Guide/CSS/Getting_started/JavaScript
https://developer.mozilla.org/it/docs/Conoscere_i_CSS
<http://www.html.it/pag/32401/html-css-e-javascript/>
- **AJAX moderno (MDN)**
- XMLHttpRequest: developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest
- Fetch API: developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- Promise & async/await: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises